



Actividad 3

Corte, Negación y Recursión en Prolog

UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA EN CIENCIAS Y SISTEMAS



3.1

DEFINICIÓN DE CONCEPTOS

Definición 3.1.1

Corte (!): Predicado de control en Prolog que detiene el backtracking, permitiendo optimizar o alterar el flujo de ejecución.

Definición 3.1.2

Corte Verde: Tipo de corte que mejora la eficiencia sin alterar la lógica del programa.

Definición 3.1.3

Corte Rojo: Tipo de corte que afecta la lógica del programa al forzar decisiones, debe usarse con precaución.

Definición 3.1.4

Negación como falla (+): Técnica en Prolog donde algo se considera falso si no puede ser probado como verdadero.

Definición 3.1.5

Recursión en Prolog: Mecanismo para iterar sobre estructuras como listas mediante llamadas a sí mismo.

Definición 3.1.6

Recursión de cola: Forma optimizada de recursión donde la llamada recursiva es la última operación, mejorando el uso de memoria.



3.2

COMPETENCIA(S) DEL EJEMPLO A IMPARTIR

Competencia 3.2.1

Desarrolla hechos, reglas, expresiones y predicados recursivos mediante el uso de cláusulas, ciclos, listas, unificación y cortes en Tau Prolog para modelar bases de conocimiento y resolver

3.3

SOFTWARE O MATERIALES NECESARIOS

Descripción

Se utilizará un entorno web con Tau-Prolog para cargar y ejecutar reglas que usen corte, negación y recursión.

Herramientas

- Librería de Tau-Prolog
- Editor de código (VSCode, Sublime, etc.)
- Navegador web

Codigo / Procedimiento

https://github.com/robinbuezo11/IA1_2025

Resultados Esperados



Clase 3 - Lógica e Inferencia

Cargar archivo de código Prolog [Seleccionar archivo](#) Sin archivos seleccionados

Ingrese su consulta en Prolog

Ejecutar

Agregar Nuevos Hechos

[Ejemplo: depredador\(tigre, v\)](#) [Agregar](#)

Descargar Código Actualizado

[Descargar Código](#)

3.4

EJERCICIOS EN CLASE

Descripción

- Implementar el predicado `maximo(X, Y, M)` utilizando corte verde.
- Crear el predicado `clasifica(X, Resultado)` usando corte rojo como se vio en clase.
- Escribir un predicado recursivo `suma_lista(Lista, Suma)` y su versión optimizada con recursión de cola.
- Usar `\+` para verificar si un elemento no pertenece a una lista.
- Mostrar mediante ejemplo cómo la negación como falla puede generar respuestas incorrectas con variables no instanciadas.

Entrega

Subir en UEDI un archivo `.pl` y un documento PDF con:

- Capturas de pantalla de consultas exitosas
- Explicación de cada predicado
- Análisis sobre cuándo usar corte y negación
- Comparación entre recursión directa y de cola

3.5

VALORES Y ACTITUDES



Aplicación de Valores

- **Responsabilidad:** Entrega puntual y organizada de archivos.
- **Excelencia:** Uso correcto de corte y negación según el contexto.
- **Ética:** Reconocimiento de fragmentos tomados de ejemplos o fuentes.
- **Compromiso:** Participación activa en la implementación.
- **Razonamiento lógico:** Justificación del uso del tipo de corte elegido.

